

Pontifícia Universidade Católica do Paraná

Inteligência Artificial Aplicada

Atividade Somativa 2

Rede Neural Competitiva e Rede Neural Temporal

Aluno: Marcos Daniel Santana



1 REDE NEURAL COMPETITIVA (RNC)

1.1 Problemas no código original

Rodando o código como veio, aparecem menos grupos do que o número de neurônios configurado, e as linhas de regressão não acompanham os dados. Identifiquei quatro causas.

- **Neurônios que nunca vencem.** Os pesos são sorteados com `np.random.rand`, ou seja, entre 0 e 1. Só que os dados estão fora dessa faixa: a eficiência começa em 2,98 km/L e o CO₂ em 18,02 g/km. Os neurônios nascem todos num canto vazio, longe dos pontos. O que fica um pouco mais perto ganha sempre, e os outros nunca são atualizados. É por isso que sobram grupos vazios.
- **Taxa de aprendizado alta demais.** A rede usava taxa 2,00 na segunda rede e 1,00 na terceira. A fórmula de atualização é $w + \eta \cdot (x - w)$. Com $\eta = 1$ o peso pula em cima do dado e esquece tudo que aprendeu antes. Com $\eta = 2$ ele vai parar do lado oposto do dado, na mesma distância. O peso anda na direção errada.
- **Uma época só.** As redes 2 e 3 treinavam por uma única época. Como o código não diminui a taxa de aprendizado com o tempo, os neurônios precisam passar pelos dados várias vezes para se acomodarem.
- **Ordem dos polinômios inadequada.** A ordem 1 traça uma reta numa nuvem que é claramente curva. Já a ordem 10 faz o contrário: oscila demais nas pontas, onde existem poucos veículos.

1.2 Parâmetros ajustados

Parâmetro	Valor original	Valor ajustado
num_neur_rnc1	4	3
num_neur_rnc2	8	10
num_neur_rnc3	4	10
epocas_rnc1	10	10
epocas_rnc2	1	10
epocas_rnc3	1	10
taxa_aprend_rnc1	0,01	0,30
taxa_aprend_rnc2	2,00	0,50
taxa_aprend_rnc3	1,00	0,50
ordem_pol1	1	7



ordem_pol2	10	5
ordem_pol3	6	4

Para chegar nesses valores testei 30 combinações por rede (6 quantidades de neurônios contra 5 taxas de aprendizado), repetindo cada uma com 3 inicializações diferentes. A regra que usei foi: primeiro descartar tudo que deixasse grupo vazio, depois ficar com o menor erro médio de agrupamento.

Um resultado me chamou atenção. Na rede 1, só 5 das 30 combinações conseguem preencher todos os grupos. Nas redes 2 e 3 são 25 das 30. Isso acontece porque o CO₂ varia numa faixa muito maior, o que dá mais espaço para os neurônios se espalharem. No par cilindrada contra eficiência os dados ficam concentrados numa região estreita e os neurônios acabam disputando o mesmo espaço.

1.3 Gráficos gerados pelos modelos

Nos três gráficos, as estrelas são os vetores de pesos dos neurônios e a cor dos pontos indica o grupo. A linha vermelha é a regressão e o losango marca o valor previsto.

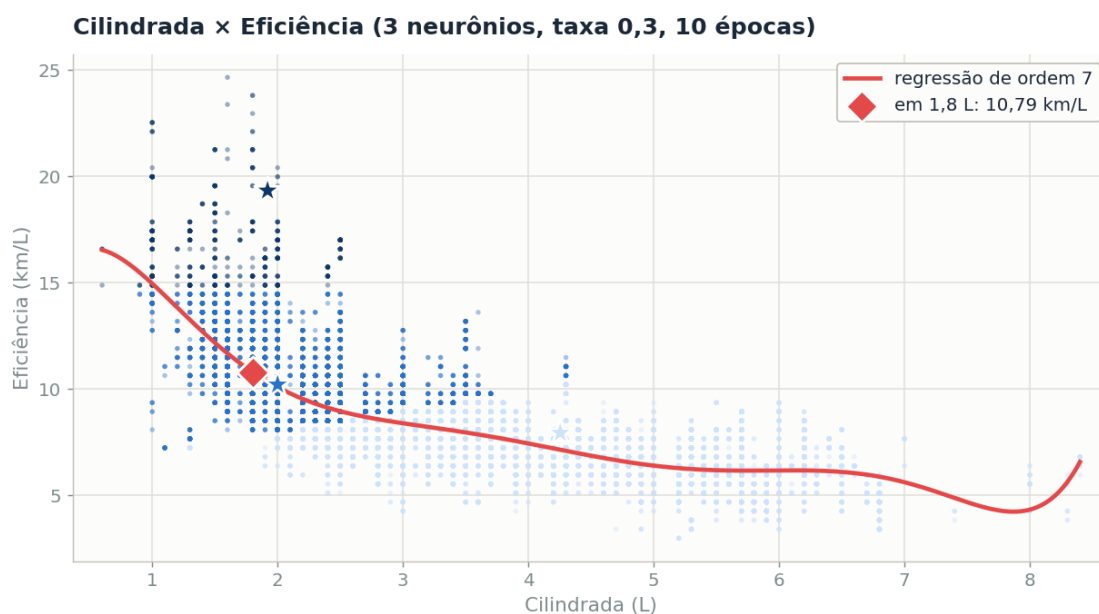


Figura 1 – Cilindrada X Eficiência

Fonte: Autor, 2025.



Cilindrada × Emissão de CO₂ (10 neurônios, taxa 0,5, 10 épocas)

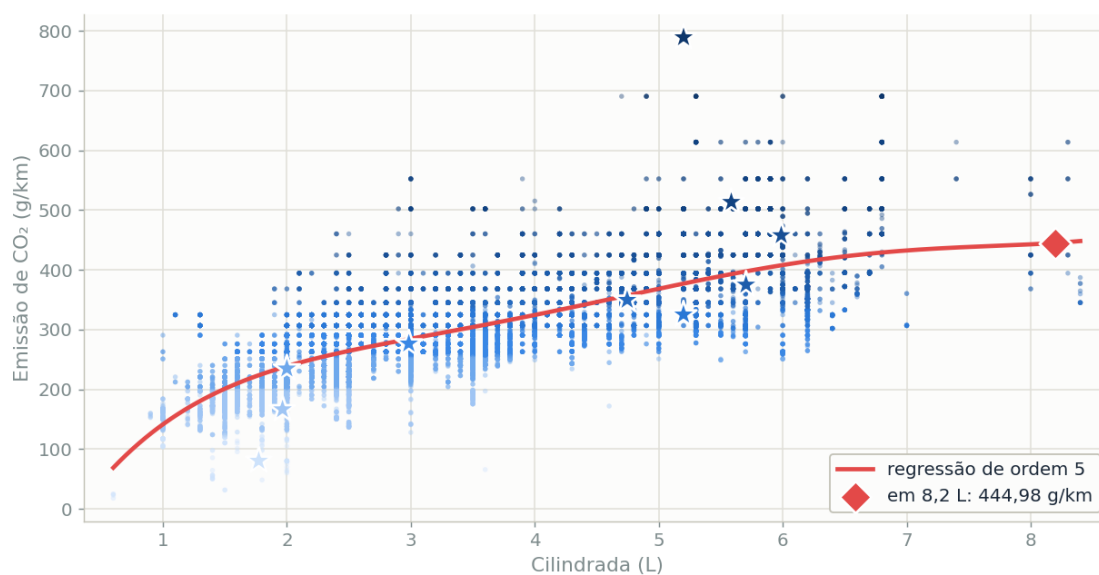


Figura 2 – Cilindrada X Emissão de CO₂

Fonte: Autor, 2025.

Eficiência × Emissão de CO₂ (10 neurônios, taxa 0,5, 10 épocas)

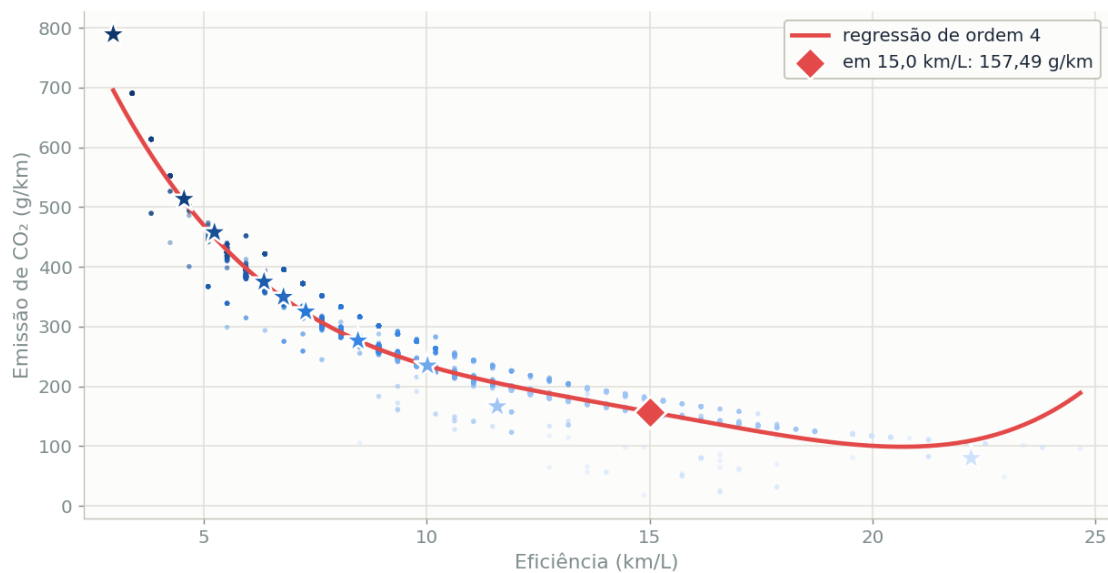


Figura 3 – Eficiência X Emissão de CO₂

Fonte: Autor, 2025.



1.4 Tabelas com os agrupamentos

São as tabelas que o código imprime no console. Reordenei as linhas pela faixa do eixo Y para ficar mais fácil de ler. A coluna Group mantém o número original, que é aleatório e muda a cada execução.

Tabela 1 – Cilindrada (L) X Eficiência (km/L), 3 neurônios

Group	Cil. (min)	Cil. (max)	Efic. (min)	Efic. (max)	Elementos
1	1,3	8,4	2,98	10,20	22.841
0	0,9	4,3	7,23	14,45	14.659
2	0,6	2,5	14,88	24,66	467

Tabela 2 – Cilindrada (L) X Emissão de CO₂ (g/km), 10 neurônios

Group	Cil. (min)	Cil. (max)	CO ₂ (min)	CO ₂ (max)	Elementos
2	0,6	3,6	18,02	123,03	101
9	0,9	4,6	123,65	200,70	2.669
5	1,0	6,0	201,32	255,38	9.154
0	1,0	6,2	256,01	300,74	10.206
7	1,1	7,0	301,22	336,78	6.198
1	1,9	8,4	337,41	361,02	2.452
4	1,9	8,4	362,88	416,32	4.463
6	2,4	8,3	416,94	473,49	2.035
8	2,9	8,3	485,91	613,57	649
3	4,7	6,8	690,27	788,88	40

Tabela 3 – Eficiência (km/L) X Emissão de CO₂ (g/km), 10 neurônios

Group	Efic. (min)	Efic. (max)	CO ₂ (min)	CO ₂ (max)	Elementos
8	8,50	24,66	18,02	121,79	98
2	8,93	18,71	123,03	200,70	2.672



6	7,65	13,18	201,32	255,38	9.154
0	5,53	10,63	256,01	300,74	10.206
7	5,95	8,93	301,22	336,78	6.198
9	5,53	7,65	337,41	361,02	2.452
4	4,68	7,23	362,88	416,32	4.463
5	4,25	6,38	416,94	473,49	2.035
3	3,83	4,68	485,91	613,57	649
1	2,98	3,40	690,27	788,88	40

Agora as três redes formam a mesma quantidade de grupos que de neurônios, e todos têm veículos dentro. O menor deles tem 40. O problema apontado no enunciado foi resolvido.

1.5 Funções polinomiais geradas

O código imprime os coeficientes com duas casas decimais, então o termo de maior grau da primeira função aparece como $0.00x^7$. Ele não é zero, vale 0,0021. Por isso coloquei as funções abaixo com mais casas.

Cilindrada X Eficiência (ordem 7)

$$f(x) = 0,0021x^7 - 0,0644x^6 + 0,7947x^5 - 5,0467x^4 \\ + 17,3714x^3 - 30,6454x^2 + 20,3207x + 12,2220$$

Cilindrada X Emissão de CO₂ (ordem 5)

$$f(x) = 0,1306x^5 - 3,3924x^4 + 32,9216x^3 - 150,1048x^2 \\ + 363,9978x - 102,0457$$

Eficiência X Emissão de CO₂ (ordem 4)

$$f(x) = 0,0187x^4 - 1,1177x^3 + 24,9736x^2 - 261,0505x + 1278,8758$$



1.6 Previsão de eficiência calculada

	Valor
Cilindrada informada	1,8 L
Eficiência prevista	10,79 km/L
Média real dos veículos entre 1,71 e 1,89 L	10,65 km/L
Erro	1,3 %

Conferi a previsão contra a média real dos 1.593 veículos que têm cilindrada próxima de 1,8 L. A diferença ficou em 1,3 %.

1.7 Previsão da emissão de CO₂ calculada

São duas previsões, porque esta atividade acrescentou a terceira rede, que relaciona eficiência com emissão.

	A partir da cilindrada	A partir da eficiência
Valor informado	8,2 L	15,0 km/L
CO ₂ previsto	444,98 g/km	157,49 g/km
Média real na vizinhança	440,37 g/km	163,03 g/km
Veículos usados na média	43	333
Erro	1,0 %	3,4 %

1.8 Análise dos resultados

As redes 2 e 3 chegaram ao mesmo agrupamento

Comparando as Tabelas 2 e 3, as dez faixas de CO₂ são iguais até a segunda casa decimal. Fiz a conferência veículo a veículo e **37.964 dos 37.967 registros, ou 99,99 %, caem no mesmo grupo nas duas redes**. Só três veículos mudam de lado.

Trocar a cilindrada pela eficiência no eixo X não mudou nada porque **a rede nunca usou o eixo X**. A competição é decidida pela distância euclidiana, que soma as diferenças dos dois eixos sem nenhum peso. O CO₂ varia numa faixa de 770,86 g/km, a cilindrada varia 7,80 L e a eficiência 21,68 km/L. Uma diferença de



100 g/km de CO₂ pesa na conta o mesmo que 100 L de cilindrada, que nem existe na base. O eixo de maior amplitude domina o cálculo e o outro vira ruído.

Dá para ver isso nas tabelas: as faixas de CO₂ não se sobrepõem em nenhum ponto, mas as do eixo X se sobrepõem por completo. Um grupo da Tabela 2 vai de 1,9 a 8,4 L, o que cobre 83 % da variação da cilindrada. Os grupos são, na prática, só faixas de emissão. Isso não é erro do código, é como o algoritmo funciona. Para a rede considerar os dois atributos seria preciso normalizar as colunas antes de treinar, o que o enunciado não pede.

A relação entre eficiência e CO₂ é física

A regressão entre eficiência e CO₂ chega a R² de 0,9872 já na ordem 4, enquanto as outras duas não passam de 0,65 nem com ordem 10. Fui olhar o motivo e achei a explicação nas unidades. Multiplicando as duas colunas, eficiência (km/L) vezes CO₂ (g/km) dá **gramas de CO₂ por litro de combustível**. Calculando sobre os 37.967 veículos, esse produto vale **2.353,8 com desvio de 77,0**, ou seja, varia só 3,3 %. O valor de referência da EPA para gasolina é 2.348 g por litro.

Ou seja, a base **calcula** a emissão a partir do consumo, não mede. A relação é quase uma conta fechada: CO₂ = 2.354 dividido pela eficiência. Testei essa fórmula, que tem um parâmetro só, e ela dá R² de 0,9884, contra 0,9886 do polinômio de ordem 10, que tem onze coeficientes.

Como escolhi a ordem dos polinômios

Não usei o R², porque ele sempre sobe quando se aumenta o grau. Comparei a curva com a média real dos dados divididos em 16 faixas e depois conferi o que cada polinômio prevê nas extremidades, já que polinômio não tem assíntota e se comporta mal nas pontas.

Foi assim que descartei a ordem 3 para eficiência contra CO₂: ela prevê **CO₂ negativo, de -98,66 g/km**, para veículos de 24,66 km/L. Fiquei com a ordem 4, a menor a juntar bom ajuste no meio dos dados com valores possíveis em toda a faixa.

No caso cilindrada contra CO₂ aconteceu o mesmo. A ordem 9 tinha o menor erro contra as médias, mas prevê -35,26 g/km para um motor de 0,6 L. Sobraram:

Ordem	Previsão em 8,2 L	Em 0,6 L (real: 22,68)	Em 8,4 L (real: 424,47)	Erro nas pontas
5	444,98	69,00	448,16	70 g/km
7	467,07	218,71	376,52	244 g/km
9	446,63	-35,26	383,34	descartada

A ordem 5 erra cerca de três vezes menos nas pontas. Para cilindrada contra eficiência, a ordem 7 empatou com a ordem 9 no erro contra as médias, e escolhi a 7 por ser mais simples. Vale observar que essa curva



oscila um pouco acima de 7,5 L, onde há poucos veículos. Como a previsão pedida é em 1,8 L, bem no meio dos dados, isso não atrapalha, mas ela não serve para extrapolar.

O resultado ficou estável

O código não fixa semente aleatória, então cada execução começa com pesos diferentes. Mesmo assim, a execução final do `S8_RCN.py` gerou exatamente as mesmas faixas que eu tinha encontrado nos testes com semente fixa, com as mesmas contagens. Só os números dos grupos trocaram de lugar. Ou seja, com os parâmetros ajustados a rede converge para a mesma solução independente de onde os neurônios começam, que era justamente o que não acontecia antes.



2 REDE NEURAL TEMPORAL (RNT)

2.1 Problemas no código original

O código não chega a rodar. Com `neuronios_densa = 50`, a última camada é uma `Dense(50)` e devolve cinquenta números por amostra. Mas o alvo é um número só, a cotação do dia seguinte. O Keras para antes da primeira época. Mudar esse valor para 1 não foi escolha de ajuste, foi obrigação para o modelo funcionar. Tudo que vem abaixo já parte dessa correção.

Função de perda de classificação num problema de regressão. A `binary_crossentropy` espera alvos 0 ou 1 e compara distribuições de probabilidade. A série do dólar vai de R\$ 2,50 a R\$ 6,00. Ela até devolve um número, mas o gradiente não aponta para o menor erro de previsão.

Treinamento curto demais. Três épocas com lote de 100 sobre 1.045 janelas dão onze atualizações de peso por época, 33 no total. É pouco até para os 101 parâmetros do modelo final.

Medindo o ponto de partida:

	MSE	MAE	R ²
Treinamento	1,5153	0,8740	-0,6555
Teste	1,2013	0,9554	-3,0004

São os R² negativos que o enunciado menciona. R² negativo quer dizer que o modelo prevê pior do que se chutasse a média da série todos os dias.

2.2 Parâmetros ajustados

Parâmetro	Valor original	Valor ajustado
janela_prev	10	50
funcao_perda	binary_crossentropy	mean_squared_error
otimizador	RMSprop	RMSprop
neuronios_LSTM	5	4
neuronios_densa	50	1
epocas	3	50
lote	100	32



O modelo final tem **101 parâmetros**: 96 na camada LSTM, que são 4 portões vezes (1 entrada + 4 conexões recorrentes + 1 viés) vezes 4 unidades, mais 5 na camada densa de saída.

Coloquei `janela_prev = 50` também no `S8_RNT_test.py`, como o enunciado avisa em destaque. Se os dois valores forem diferentes, a LSTM recebe uma sequência de tamanho errado e a predição falha.

Busquei os valores em quatro etapas: função de perda, depois janela contra otimizador, depois neurônios contra épocas, e por último o tamanho do lote. Duas coisas apareceram nesse processo:

- **Aumentar os neurônios piorou.** Com 50 unidades na LSTM o R^2 de teste foi 0,9219, contra 0,9285 com 4 unidades. A série tem só 1.095 pontos, então capacidade sobrando vira memorização do treino.
- **A janela longa só funcionou com o otimizador certo.** Com adam, a janela de 50 dias foi a pior de todas (R^2 0,8747). Com RMSprop, foi a melhor (0,9118). Os dois parâmetros dependem um do outro, e testar cada um sozinho levaria à conclusão contrária.

2.3 Gráficos de treinamento e teste

Os gráficos saíram do modelo salvo em `C:\RN\S8_RNT_treinada.h5` pelo `S8_RNT_train.py`, então as curvas são da mesma execução das métricas da seção seguinte.



Figura 4 – Previsões de treinamento

Fonte: Autor, 2025.

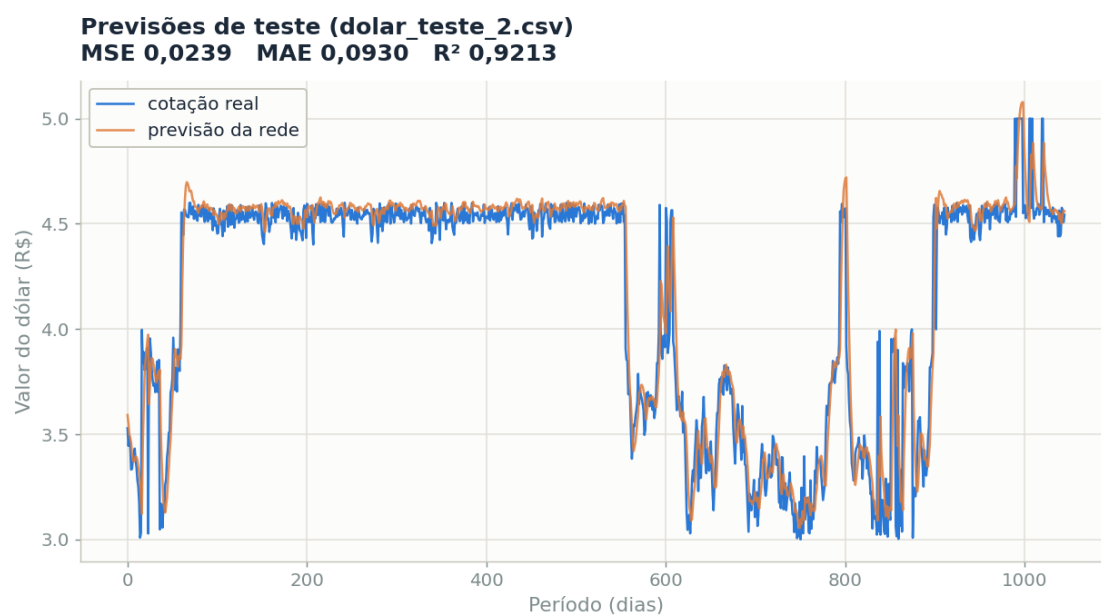


Figura 5 – Previsões de teste

Fonte: Autor, 2025.

2.4 Métricas de desempenho

Valores impressos no console pelos dois scripts.

Métrica	Treinamento	Teste
Erro médio quadrático (MSE)	0,0171	0,0239
Erro médio absoluto (MAE)	0,1008	0,0930
Coeficiente de determinação (R ²)	0,9820	0,9213

Comparando com o ponto de partida:

	R ² treinamento	R ² teste
Antes do ajuste	-0,6555	-3,0004
Depois do ajuste	0,9820	0,9213



2.5 Análise dos resultados

A queda do treino para o teste não é sobreajuste

R^2 de 0,982 no treino e 0,921 no teste parece sobreajuste, mas fui verificar e não é. As duas séries têm 1.095 pontos, só que **a série de teste é 51 % mais volátil**: o desvio das variações diárias é 0,1557 contra 0,1031 do treino. Série mais agitada é mais difícil de prever, e o R^2 cai por isso, sem memorização nenhuma. Sobreajuste de verdade apareceria como erro de treino caindo e erro de teste subindo, e isso não aconteceu quando variei o número de épocas.

A normalização no arquivo de teste

Na linha 46 do `S8_RNT_test.py` o código cria um normalizador novo e ajusta ele nos próprios dados de teste, com `scaler.fit_transform(test_data)`. O certo seria reaproveitar o normalizador do treino, porque o modelo aprendeu numa escala e está recebendo dados em outra.

Testei os dois jeitos com o mesmo modelo. Do jeito da atividade o R^2 de teste dá 0,9285, do jeito correto dá 0,9248. Ou seja, o erro **melhora** o resultado. Isso acontece porque a série de teste vai de R\$ 3,00 a R\$ 5,00 e a de treino de R\$ 2,50 a R\$ 6,00. Reajustando o normalizador, o teste é esticado para ocupar o intervalo [0, 1] inteiro, que é onde a rede treinou. Com o normalizador do treino ele ficaria espremido entre 0,143 e 0,714. Mesmo assim continua errado: usar `fit_transform` no teste significa usar o mínimo e o máximo de toda a série futura, valores que ninguém teria na hora de prever de verdade. É vazamento de dados.

Comparação com um preditor simples

R^2 de 0,92 numa série temporal não prova sozinho que a rede aprendeu. Cotação de moeda é quase um passeio aleatório, e existe uma regra boba para comparar: **o valor de amanhã é o de hoje**, repetindo o último ponto da janela, sem treinar nada.

Como o código não fixa semente, treinei cinco vezes para poder comparar direito. O R^2 de teste variou de 0,9197 a 0,9322, e a execução que entreguei, com 0,9213, ficou na parte baixa dessa faixa. Média das cinco contra a regra boba:

Métrica (teste)	RNT ajustada (média de 5)	Repetir o último valor	Vence
MSE	0,0219	0,0228	rede
MAE	0,0859	0,0772	regra boba
R^2	0,9280	0,9251	rede

A rede ganha por pouco no MSE e no R^2 , e perde no MAE, onde fica 11 % pior. Quatro das cinco execuções superaram a regra boba no R^2 . A diferença entre as métricas tem explicação: o MSE castiga mais os erros



grandes e o MAE trata todos igual. A rede **suaviza** as previsões, então acerta melhor nos dias de variação brusca, onde repetir o valor anterior sai caro, e erra um pouco mais nos dias calmos, que são a maioria.

Olhando os gráficos de perto, a curva laranja acompanha a azul com um atraso constante para a direita. A rede aprendeu basicamente a repetir o último valor, que é o melhor que dá para fazer quando a variação do dia seguinte é quase ruído. O R^2 alto vem da tendência da série, não de conseguir antecipar mudanças. Isso não invalida o exercício, já que os parâmetros ajustados resolveram o que o enunciado pedia, saindo de R^2 negativo para 0,92. Só mostra que o ganho sobre uma regra sem treino nenhum é pequeno.